



Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Інженерія програмного забезпечення

Практична робота №2

«Графічна нотація UML. Документування проекту»

Виконав: студент 2 курсу ФІОТ, гр. ІО-41

Давидчук А. М.

Залікова книжка № 4106

Перевірив: *Васильєва М. Д.*

Тема: «Графічна нотація UML. Документування проекту».

Мета: Ознайомитись з видами UML діаграм. Отримати базові навички з використання діаграми класів мови UML. Здобути навички використання засобів автоматизації UML-моделювання. Документування проекту за допомогою JavaDoc.

Практична частина

Мій номер залікової книги 4106, звідси $4106 \bmod 9 = 2$, і $4106 \bmod 5 = 1$. Звідси згідно переліку варіантів завдання, моя схема генералізації (наслідування):

If1 $\leftarrow$ If2; If1 $\leftarrow$ If3; Cl2 $\leftarrow$ Cl3

і агрегації:

If1 $\leftarrow$ Cl2; Cl1 $\leftarrow$ Cl3; Cl1 $\leftarrow$ Cl1

Звідси побудую UML-діаграму класів, яка відображає взаємозв'язки між трьома інтерфейсами (If1, If2, If3) та трьома класами (Cl1, Cl2, Cl3) згідно з моїм варіантом. Кожен інтерфейс містить методи void meth1(), void meth2(), void meth3(), а відповідний клас реалізує їх:

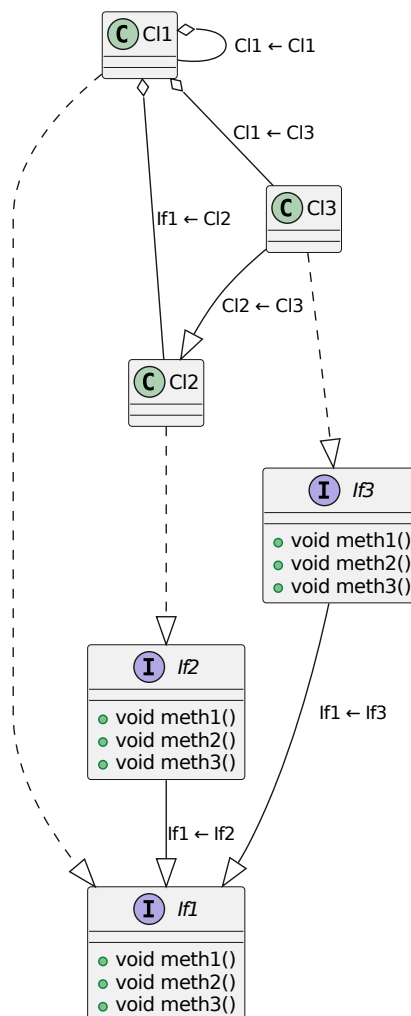


Рис. 1: UML-діаграма класів та інтерфейсів

Програмний код (project1\src\lab2\Main.java):

```
1 package lab2;
2
3 /**
4  * Клас Main призначений для тестування реалізованих класів і інтерфейсів.
5  * Демонструє виклики методів і створення агрегованих зв'язків.
6  * @author Artem Davydchuk
7  * @version 1.0
8  */
9
10 public class Main {
11     /**
12      * Точка входу до програми.
13      * @param args аргументи командного рядка
14      */
15     public static void main(String[] args) {
16         Cl1 c1 = new Cl1();
17         Cl2 c2 = new Cl2();
18         Cl3 c3 = new Cl3();
19
20         // Агрегація
21         c1.setPart2(c2);
22         c1.setPart3(c3);
23         c1.addChild(new Cl1());
24
25         // Виклик методів
26         c1.meth1();
27         c2.meth2();
28         c3.meth3();
29     }
30 }
```

Код інтерфейсу If1 (project1\src\lab2\If1.java):

```
1 package lab2;
2
3 /**
4  * Базовий інтерфейс If1.
5  * Містить три методи meth1(), meth2(), meth3().
6  * Використовується як основа для If2 і If3.
7  * @author Artem Davydchuk
8  * @version 1.0
9  */
10
11 public interface If1 {
12     /**
13      * Перший метод інтерфейсу If1.
14      */
15     void meth1();
16
17     /**
18      * Другий метод інтерфейсу If1.
19      */
20 }
```

```

20     void meth2();
21
22     /**
23      * Третій метод інтерфейсу If1.
24      */
25     void meth3();
26 }
27

```

Код інтерфейсу If2 (project1\src\lab2\If2.java):

```

1  package lab2;
2
3  /**
4   * Інтерфейс If2 успадковує If1.
5   * Призначений для розширення функціональності базового інтерфейсу.
6   * @author Artem Davydchuk
7   * @version 1.0
8   */
9
10 public interface If2 extends If1 {}

```

Код інтерфейсу If3 (project1\src\lab2\If3.java):

```

1  package lab2;
2
3  /**
4   * Інтерфейс If2 успадковує If1.
5   * Призначений для розширення функціональності базового інтерфейсу.
6   * @author Artem Davydchuk
7   * @version 1.0
8   */
9
10 public interface If3 extends If1 {}

```

Код класу Cl1 (project1\src\lab2\Cl1.java):

```

1  package lab2;
2
3  import java.util.ArrayList;
4  import java.util.List;
5
6  /**
7   * Клас Cl1 реалізує інтерфейс If1.
8   * Містить поля-агрегації: об'єкти Cl2, Cl3 і список власних екземплярів
9   * ↪ (self-aggregation).
10  * Кожен метод виводить у консоль своє ім'я для демонстрації викликів.
11  * @author Artem Davydchuk
12  * @version 1.0
13  */
14 public class Cl1 implements If1 {
15     /** Агрегований об'єкт типу Cl2.

```

```

15     * @uml.aggregation */
16     private C12 part2;
17
18     /** Агрегований об'єкт типу C13.
19      * @uml.aggregation */
20     private C13 part3;
21
22     /** Список підлеглих екземплярів C11 (самоагрегація). */
23
24     private List<C11> children = new ArrayList<>();
25
26     /**
27      * Джерело опису: {@link If1#meth1()}.
28      * {@inheritDoc}
29      */
30     @Override
31     public void meth1() { System.out.println("C11: meth1"); }
32
33     /**
34      * Джерело опису: {@link If1#meth2()}.
35      * {@inheritDoc}
36      */
37     @Override
38     public void meth2() { System.out.println("C11: meth2"); }
39
40     /**
41      * Джерело опису: {@link If1#meth3()}.
42      * {@inheritDoc}
43      */
44     @Override
45     public void meth3() { System.out.println("C11: meth3"); }
46
47     /**
48      * Задає агрегований об'єкт типу C12.
49      * @param p екземпляр C12
50      */
51     public void setPart2(C12 p) { this.part2 = p; }
52
53     /**
54      * Задає агрегований об'єкт типу C13.
55      * @param p екземпляр C13
56      */
57     public void setPart3(C13 p) { this.part3 = p; }
58
59     /**
60      * Додає дочірній екземпляр C11 до списку.
61      * @param child підлеглий об'єкт C11
62      */
63     public void addChild(C11 child) { this.children.add(child); }
64 }

```

Код класу C12 (project1\src\lab2\C12.java):

```

1 package lab2;

```

```

2
3 /**
4  * Клас Cl2 реалізує інтерфейс If2.
5  * Реалізує всі методи базового інтерфейсу If1.
6  * @author Artem Davydchuk
7  * @version 1.0
8  */
9
10 public class Cl2 implements If2 {
11     /**
12      * Джерело опису: {@link If1#meth1()}
13      * (через {@link If2}).
14      * {@inheritDoc}
15      */
16     @Override
17     public void meth1() { System.out.println("Cl2: meth1"); }
18
19     /**
20      * Джерело опису: {@link If1#meth2()} (через {@link If2}).
21      * {@inheritDoc}
22      */
23     @Override
24     public void meth2() { System.out.println("Cl2: meth2"); }
25
26     /**
27      * Джерело опису: {@link If1#meth3()} (через {@link If2}).
28      * {@inheritDoc}
29      */
30     @Override
31     public void meth3() { System.out.println("Cl2: meth3"); }
32 }

```

Код класу Cl3 (project1\src\lab2\Cl3.java):

```

1 package lab2;
2
3 /**
4  * Клас Cl3 успадковує Cl2 та реалізує інтерфейс If3.
5  * Реалізує всі методи з виведенням у консоль.
6  * @author Artem Davydchuk
7  * @version 1.0
8  */
9
10 public class Cl3 extends Cl2 implements If3 {
11     /**
12      * Джерело опису: {@link Cl2#meth1()} (транзитивно з {@link If1#meth1()}).
13      * {@inheritDoc}
14      */
15     @Override
16     public void meth1() { System.out.println("Cl3: meth1"); }
17
18     /**
19      * Джерело опису: {@link Cl2#meth2()} (транзитивно з {@link If1#meth2()}).
20      * {@inheritDoc}

```

```

21     */
22     @Override
23     public void meth2() { System.out.println("Cl3: meth2"); }
24
25     /**
26     * Джерело опису: {@link Cl2#meth3()} (транзитивно з {@link If1#meth3()}).
27     * {@inheritDoc}
28     */
29     @Override
30     public void meth3() { System.out.println("Cl3: meth3"); }
31 }
32

```

Код збірки для Ant (project1\src\build.xml):

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <project name="Lab2_Project1" default="all" basedir=".">
3
4     <!-- ===== Налаштування шляхів ===== -->
5     <property name="src.dir" value="src"/>
6     <property name="build.dir" value="out"/>
7     <property name="doc.dir" value="doc"/>
8     <property name="main.class" value="lab2.Main"/>
9
10    <!-- ===== 1. Компіляція Java-файлів ===== -->
11    <target name="compile" description="Компіляція вихідних Java-файлів у теку out">
12        <mkdir dir="${build.dir}"/>
13        <javac
14            includeantruntime="false"
15            encoding="UTF-8"
16            srcdir="${src.dir}"
17            destdir="${build.dir}"
18        </javac>
19        <echo message="Компіляція завершена. Клас-файли збережено у
20            ↳ "${build.dir}"/>
21    </target>
22
23    <!-- ===== 2. Генерація JavaDoc ===== -->
24    <target name="javadoc" description="Створення HTML-документації в каталозі doc">
25        <mkdir dir="${doc.dir}"/>
26        <!-- additionalparam="-Xdoclint:none -quiet" Щоб видалити попередження про
27            ↳ конструктор класів -->
28        <javadoc
29            sourcepath="${src.dir}"
30            destdir="${doc.dir}"
31            encoding="UTF-8"
32            docencoding="UTF-8"
33            charset="UTF-8"
34            author="true"
35            version="true"
36            use="true"
37            additionalparam="-Xdoclint:none -quiet"
38            windowtitle="JavaDoc - Lab2 Project1"
39            doctitle="Документація до лабораторної роботи №2 - UML">

```

```

38         <fileset dir="${src.dir}" includes="**/*.java"/>
39     </javadoc>
40     <echo message="JavaDoc успішно згенеровано у каталозі '${doc.dir}'."/>
41 </target>
42
43 <!-- ===== 3. Запуск програми ===== -->
44 <target name="run" depends="compile" description="Запуск програми після
↳ компіляції">
45     <echo message="Запуск програми ${main.class}..."/>
46     <java classname="${main.class}" fork="true">
47         <classpath>
48             <pathelement path="${build.dir}"/>
49         </classpath>
50     </java>
51     <echo message="Програму завершено."/>
52 </target>
53
54 <!-- ===== 4. Об'єднана команда ===== -->
55 <target name="all" depends="compile, javadoc, run" description="Компіляція +
↳ JavaDoc + запуск">
56     <echo message="Проект повністю зібрано, задокументовано та виконано."/>
57 </target>
58
59 </project>

```

Тут build.xml має такі команди збірки:

```

1 PS C:\Users\artem\OneDrive\Desktop\reps\university_roadmap\semestr3\software
↳ engineering\labs\lab2\project1> ant -p
2 Buildfile: C:\Users\artem\OneDrive\Desktop\reps\university_roadmap\semestr3\software
↳ engineering\labs\lab2\project1\build.xml
3
4 Main targets:
5
6 all      Комп?ляц?я + JavaDoc + запуск
7 compile  Комп?ляц?я вих?дних Java-файл?в у теку out
8 javadoc   Створення HTML-документац?ї в каталоз? doc
9 run      Запуск програми п?сля комп?ляц?ї

```

Виконуємо збірку (команда all є default-командою):

```

1 PS C:\Users\artem\OneDrive\Desktop\reps\university_roadmap\semestr3\software
↳ engineering\labs\lab2\project1> ant
2 Buildfile: C:\Users\artem\OneDrive\Desktop\reps\university_roadmap\semestr3\software
↳ engineering\labs\lab2\project1\build.xml
3
4 compile:
5     [echo] Комп?ляц?я завершена. Клас-файли збережено у 'out'.
6
7 javadoc:
8     [javadoc] Generating Javadoc
9     [javadoc] Javadoc execution
10    [echo] JavaDoc успішно згенеровано у каталоз? 'doc'.

```



```

11
12 run:
13     [echo] Запуск програми lab2.Main...
14     [java] C11: meth1
15     [java] C12: meth2
16     [java] C13: meth3
17     [echo] Програму завершено.
18
19 all:
20     [echo] Проект повністю зібрано, задокументовано та виконано.
21
22 BUILD SUCCESSFUL
23 Total time: 1 second

```

Результат відкриття файлу документації `index.html` (у браузері):

PACKAGE

CLASS

USE

TREE

INDEX

HELP

PACKAGE: DESCRIPTION | RELATED PACKAGES | CLASSES AND INTERFACES

Package lab2

package lab2

All Classes and Interfaces

Interfaces

Classes

Class	Description
C11	Клас C11 реалізує інтерфейс If1.
C12	Клас C12 реалізує інтерфейс If2.
C13	Клас C13 успадковує C12 та реалізує інтерфейс If3.
If1	Базовий інтерфейс If1.
If2	Інтерфейс If2 успадковує If1.
If3	Інтерфейс If3 успадковує If1.
Main	Клас Main призначений для тестування реалізованих класів і інтерфейсів.

PACKAGE

CLASS

USE

TREE

INDEX

HELP

PACKAGE: DESCRIPTION | RELATED PACKAGES | CLASSES AND INTERFACES

Package lab2

package lab2

All Classes and Interfaces

Interfaces

Classes

Class	Description
If1	Базовий інтерфейс If1.
If2	Інтерфейс If2 успадковує If1.
If3	Інтерфейс If3 успадковує If1.

All Classes and Interfaces

Interfaces

Classes

Class	Description
C11	Клас C11 реалізує інтерфейс If1.
C12	Клас C12 реалізує інтерфейс If2.
C13	Клас C13 успадковує C12 та реалізує інтерфейс If3.
Main	Клас Main призначений для тестування реалізованих класів і інтерфейсів.

Рис. 2: Сторінка документації коду у браузері

Далі за допомогою PlainUML Plugin згенерую UML-діаграму на основі всього робочого *.java коду:

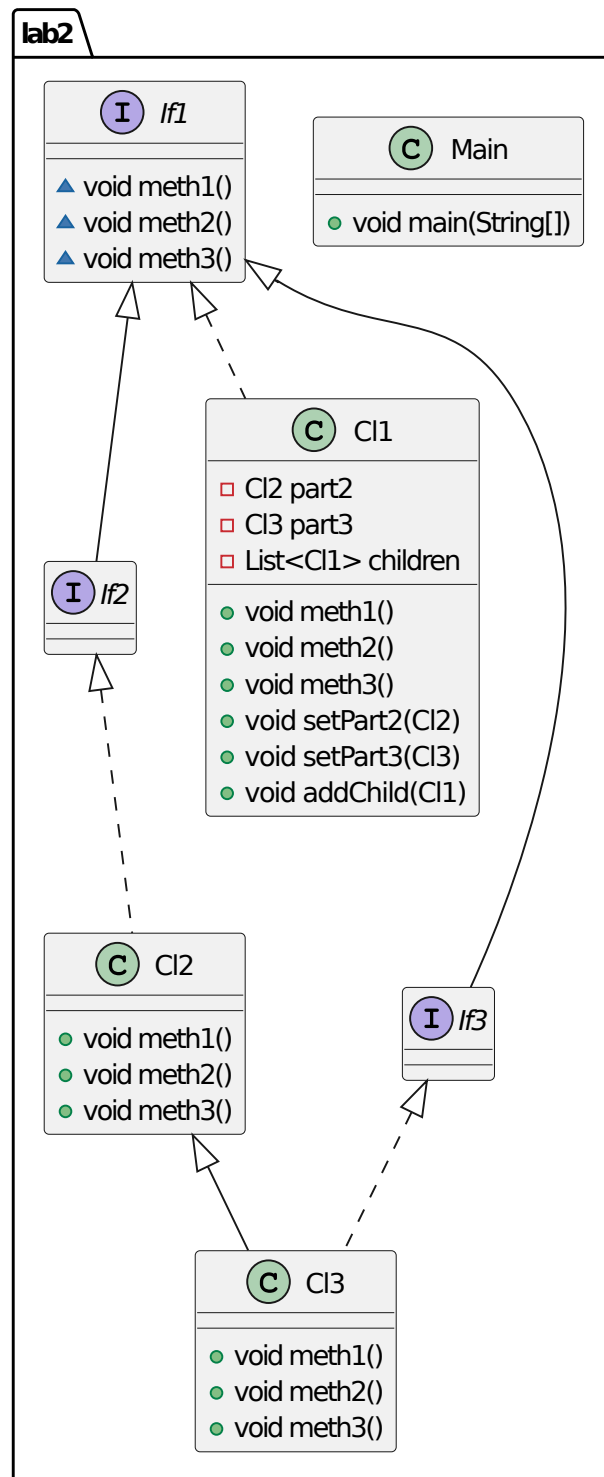


Рис. 3: Згенерована UML-діаграма на основі коду

Згенерована UML-діаграма з коду пакета `lab2` повністю відповідає розробленій вручну за складом класів та інтерфейсів. Різниця полягає у рівні деталізації: автоматично створена діаграма не дублює методи інтерфейсів у класах-реалізаціях, а зв'язки типу агрегації (`If1`←`C12`, `C11`←`C13`, `C11`←`C11`) відображено як звичайні асоціації, оскільки їх тип не може бути однозначно визначений з коду. Таким чином, обидві діаграми еквівалентні за структурою, відмінності мають лише візуальний характер.

Висновок: У ході роботи було побудовано базову UML-діаграму класів із трьома інтерфейсами та трьома класами, реалізовано зв'язки генералізації та агрегації згідно з індивідуальним варіантом, створено відповідний вихідний код мовою Java, згенеровано документацію JavaDoc та налаштовано сценарій автоматизації складання проєкту у `build.xml`. За допомогою засобів генерації UML-діаграм із вихідного коду перевірено відповідність UML-діаграми моделі програми. Згенерована діаграма повністю підтвердила коректність реалізації зв'язків та структури вручну створеної UML-діаграми.